

Git & GitHub Complete Master SOP & Rules Guide (English)

Comprehensive Operating Rules for Push, Pull, Fetch, Branching, Merging, Stashing, and Conflict Resolution

- PUSH & PULL
- FETCH VS PULL
- BRANCH & MERGE
- GIT STASH
- CONFLICT SOLVE
- MULTI-ACCOUNT SAFE

1. Repository Initialization & Initial Push

Standard sequence to initialize and publish any software project (such as `PCAccess`) to GitHub from scratch:

```
# 1. Switch to the project directory: cd /d E:\Git\PCAccess # 2. Initialize Local repository and set isolated credentials: git init git config user.name "Visibit Technologies" git config user.email "visibittechnologies@gmail.com" # 3. Create .gitignore (ignore bin/, obj/, packages/, .vs/, *.bak) # 4. Stage and create initial commit: git branch -M main git add . git commit -m "Initial commit: Project full codebase with documentation" # 5. Link Remote Origin (Multi-Account Fix: include username prefix in the URL): git remote add origin https://visibittechnologies@github.com/visibittechnologies/PCAccess-.git git push -u origin main
```

2. Fetch vs Pull: Core Differences & Operating Rules

Understanding the distinction between `fetch` and `pull` is essential for safe team collaboration:

Command	What it does	When to use
<code>git fetch origin</code>	Downloads all new commits, tags, and branches from GitHub into local tracking branches without modifying your working files.	When you want to inspect what coworkers have pushed before incorporating changes.
<code>git pull origin main</code>	Executes <code>git fetch</code> followed immediately by <code>git merge</code> . Updates your local files with remote changes.	When you are ready to update your local working branch immediately.
<code>git pull --rebase origin main</code>	Fetches remote commits and applies your local unpushed commits on top of the remote tip.	(Best Practice) Prevents unnecessary merge commits and maintains a linear commit history.

```
# Safe inspection workflow before merging: git fetch origin git log HEAD..origin/main --oneline # Review incoming remote commits git merge origin/main # Safely merge into current branch
```

3. Push Operations & Safety Guardrails

Rule 1: Initial Push with Upstream Tracking

```
git push -u origin main
```

The `-u` flag binds local `main` to remote `origin/main`. For subsequent pushes on this branch, you only need to type `git push`.

Rule 2: Daily Update Push

```
git add . git commit -m "Feature: Add remote drive reconnect handler" git push
```

Rule 3: Pushing a New Feature Branch

```
git push -u origin feature-user-agent
```

Rule 4: Handling Push Rejection (Never Blind Force Push)

If another teammate committed to GitHub before your push, Git will reject your push with *"Updates were rejected because the remote contains work that you do not have locally."*

- ❌ **Forbidden:** Never run `git push -f` (Force Push) on shared branches, as it destroys remote commit history.
- ✅ **Proper Fix:** Run `git pull --rebase origin main`, resolve any conflicts, and then run `git push`.

4. Branching & Merging (Complete Workflow)

Always isolate new features, bug fixes, or experiments into dedicated branches rather than committing directly onto `main`.

Step A: Create and Switch to a New Branch

```
# Create and immediately checkout new branch: git checkout -b feature-user-agent # (Modern syntax: git switch -c feature-user-agent)
```

Step B: Make Changes and Commit

```
git add . git commit -m "Add drive letter validation logic"
```

Step C: Merge Feature Branch into Main

```
# 1. Switch back to main branch: git checkout main # 2. Pull the latest remote main: git pull origin main  
# 3. Merge feature branch into main: git merge feature-user-agent # 4. Push updated main to GitHub: git push origin main
```

Step D: Cleanup Merged Branches

```
# Delete local branch: git branch -d feature-user-agent # Delete branch on GitHub remote: git push origin --delete feature-user-agent
```

5. Git Stash: Shelving Uncommitted Work Safely

Scenario: You are midway through modifying code, but you must urgently switch branches or pull incoming updates from GitHub. Git blocks you stating "Please commit or stash your changes".

Essential Git Stash Commands:

```
# 1. Temporarily shelve uncommitted working tree changes: git stash save "WIP: Drive mounting refactor" #  
Working directory is now clean. You can safely pull or switch: git pull origin main # 2. Restore your  
shelved changes back to the working directory: git stash pop # 3. List all saved stashes: git stash list  
# 4. Discard a specific or latest stash: git stash drop
```

6. Merge Conflict Resolution (Step-by-Step SOP)

Why do conflicts happen? When two branches have modified the exact same lines of code within the same file, Git pauses the merge and asks the developer to decide which version to preserve.

Visual Anatomy of a Merge Conflict:

```
<<<<<<< HEAD (Your Local Working Code)  
string apiUrl = "https://pcaccess.ourdemos.com";  
=====  
string apiUrl = "https://api.pcaccess.com";  
>>>>>> origin/main (Incoming Remote Code)
```

4 Steps to Resolve Any Conflict:

1. Open Conflicted Files in IDE:

Locate the <<<<<<<, =====, and >>>>>>> conflict markers.

2. Perform Manual Decision & Cleanup:

Keep the intended lines of code, remove unwanted lines, and **delete all 3 conflict delimiter lines completely.**

3. Stage Resolved Files and Finalize:

```
git add . git commit -m "Fix: Resolve merge conflict in AgentConfig" git push
```

4. Emergency Abort (Return to Clean State):

```
git merge --abort # (If conflict occurred during git pull --rebase): git rebase --abort
```

7. GitHub Desktop Operations Matrix

Operation	GitHub Desktop Action
Fetch / Pull	Click " Fetch origin " in the top right. If remote commits exist, the button becomes " Pull origin ". Click to merge.
Push	After creating a local commit, the upper bar displays " Push origin " with commit count. Click to upload.
Branch Creation	Click " Current branch " dropdown → Click " New branch " → Provide branch name → Click " Create branch ".
Branch Merging	Menu: Branch → " Merge into current branch... " → Select source branch → Click " Create a merge commit ".
Stash Changes	When switching branches with modified files, GitHub Desktop automatically asks to stash or carry changes to the target branch.
Conflict Resolution	GitHub Desktop lists all conflicted files with an " Open in Visual Studio " shortcut. Once markers are cleaned, a green checkmark appears.

8. Golden Git Master Cheat Sheet

```
# 1. Status & History git status # Check current modified files git log --oneline -n 5 # View 5 most recent commits # 2. Stashing git stash # Hide incomplete work git stash pop # Restore shelved work # 3. Pulling & Fetching git fetch origin # Download metadata without touching local files git pull --rebase origin main # Pull remote changes with clean linear history # 4. Branch Management git checkout -b feature-name # Create and switch to branch git checkout main # Return to production branch git merge feature-name # Merge branch into current branch # 5. Pushing Code git push origin main # Upload commits to GitHub
```